Here is the example's important point: "Whenever the program changes scope, some bindings are replaced by a new set of bindings." Granted, this doesn't happen to all the bindings each and every time. For example, if a method calls another method on the same object, instance variables stay in scope through the call. In general, though, bindings tend to fall out of scope when the scope changes. In particular, local variables change at every new scope. (That's why they're "local.")

As you can see, keeping track of scopes can be a tricky task. You can spot scopes more quickly if you learn about *Scope Gates*.

Scope Gates

There are exactly three places where a program leaves the previous scope behind and opens a new one:

- Class definitions
- Module definitions
- Methods

Scope changes whenever the program enters (or exits) a class or module definition or a method. These three borders are marked by the keywords `class`, `module`, and `def`, respectively. Each of these keywords acts like a *Scope Gate*.

Spell: Scope Gate

For example, here is the previous example program again, with Scope Gates clearly marked by comments:

```
v1 = 1
class MyClass        # SCOPE GATE: entering class
  v2 = 2
  local_variables    # => ["v2"]
  def my_method      # SCOPE GATE: entering def
    v3 = 3
    local_variables
  end                # SCOPE GATE: leaving def
  local_variables    # => ["v2"]
end                  # SCOPE GATE: leaving class

obj = MyClass.new
obj.my_method        # => [:v3]
local_variables      # => [:v1, :obj]
```

Now it's easy to see that this program opens three separate scopes: the top-level scope, one new scope when it enters `MyClass`, and one new scope when it calls `my_method`.

There is a subtle difference between `class` and `module` on one side and `def` on the other. The code in a class or module definition is executed immediately.